Can Deterministic Network Verification Reduce Undetected Faults in LLM-Generated Configurations?

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

***Abstract*—We preregistered and executed a paired confirmatory study (study id c1-gnr-vv-2026-0001) to measure whether inserting a deterministic network verifier between an LLM intent→configuration generator and an emulated execution reduces the proportion of configurations that would execute with operational faults. Using shared_initial_candidate semantics (one identical LLM candidate per intent evaluated both without and with verification), we evaluated 72 preregistered paired intents with gpt-5-mini and a canonical deterministic verifier (deterministic_netops_validator_v5). Execution completed with 144 episodes (72 pairs) and 72 model calls. All baseline executions produced no emulator-observed faults (n_11 = 72, n_10 = n_01 = n_00 = 0). The paired difference in undetected-fault proportion is 0.0 (paired bootstrap 95% CI [0.0, 0.0], exact McNemar two-sided p = 1.0; bootstrap seed = 1729, resamples = 10000). Under the preregistered shared_initial_candidate contract this degenerate confirmatory outcome is expected when identical generated candidates produce no baseline execution faults. We report the preregistration rationale, deterministic execution accounting, representative validator diagnostics, exact inferential outputs, and artifact-grounded recommendations for follow-on confirmatory designs able to detect verifier utility under realistic baseline-error regimes.**

I. Introduction

Generative large language models (LLMs) are increasingly proposed to translate high-level operator intents into device-level network configurations. This intent→configuration automation can speed change pipelines but risks silently violating reachability, access-control, ordering, or workflow invariants and thereby causing outages if generated text is executed without additional checks. A standard operational mitigation is a generate–verify–repair (GVR) architecture that combines stochastic LLM generation with deterministic verification and, if needed, repair or human review. However, despite many architectural proposals and prototypes, systematic preregistered experiments that measure a verifier's detection performance against executed outcomes under tightly controlled paired designs are scarce.

This paper answers a narrowly scoped, preregistered research question: does inserting a deterministic network verifier between an LLM generator and emulator execution materially reduce the proportion of configurations that execute with operational faults when the identical generated candidate is evaluated both without and with verification? That estimand isolates verifier detection from improvements that could arise from re-sampling, prompt-engineering, or repair-enabled iterations. We executed study c1-gnr-vv-2026-0001 using gpt-5-mini and deterministic_netops_validator_v5 across 72 preregistered paired intents under shared_initial_candidate semantics. The run completed with no technical failures, produced a degenerate confirmatory outcome (no baseline emulator faults), and yields a paired difference of 0.0 (95% CI [0.0,0.0], exact McNemar p = 1.0). Rather than treating this as a null failure, we analyze why it occurred given the preregistered contract and archived artifacts, and we provide concrete, artifact-grounded guidance for designs that can detect verifier utility when baseline error prevalence is non-negligible.

II. Related Work

We position this study against three closely related strands: (1) LLM-based intent→configuration automation and emulated evaluations; (2) multi-agent and tool-augmented NetOps frameworks that propose verifier integration; and (3) generate–verify–repair and constrained-decoding methods from software/code domains that inform possible NetOps adaptations.

LLM intent→configuration translation. Recent empirical work demonstrates that instruction-tuned LLMs can map operator intents to device-level configuration text within constrained vendor dialects and curated testbeds. For example, Nguyen et al. present an intent→configuration pipeline and evaluate LLM performance on structured configuration tasks, documenting promising task-level accuracy but highlighting the gap between textual correctness and safe execution in operational environments [1]. Those studies use emulator or synthesized workloads to assess functional correctness of generated artifacts but emphasize that textual matching metrics do not substitute for execution-level invariants such as reachability or ordering constraints. Our study adopts the same operational framing (emulator execution as a conservative proxy for operational fault) but differs methodologically by preregistering a paired detection estimand under shared_initial_candidate semantics to isolate verifier detection capability on identical generated candidates.

Multi-agent and verifier-integrating NetOps systems. Several prototypes and architectural reports advocate tool-augmented or multi-agent LLM systems that incorporate verification steps to reduce regressions and to enable safe automation at scale. Notably, recent multi-agent intent-driven frameworks demonstrate how separate agent roles (planner, config generator, validator) can be orchestrated to produce and check configurations before execution; these systems show

how verifier signals can be folded into human-in-the-loop or automated repair workflows but typically report system behavior at the demonstration or prototype level rather than in preregistered paired detection trials [2]. Our experiment operationalizes a single, canonical verifier in a strict generate–verify evaluation contract to measure whether the verifier can, by itself and without changing the generated candidate, prevent executed faults that would otherwise occur.

Generate–Verify–Repair and constrained decoding provenance. The generate–verify–repair (GVR) pattern has a substantial methodological pedigree in program repair and structured-output generation. Architectural proposals and empirical studies in code generation and self-repair show that deterministic checkers (unit tests, static analyzers) combined with stochastic generation can enable iterative repair loops that converge on functionally correct outputs [3]. Complementary work on constrained decoding and integer-programming-based decoding demonstrates that enforcing structural constraints during generation reduces syntactic or semantic violations in structured domains [4]. These results motivate two dimensions of adaptation for NetOps: (a) constraining or biasing generation toward verifier-satisfiable forms (constrained decoding), and (b) allowing verifier-triggered repair iterations that change the executed candidate distribution. Importantly, those adaptations change the causal estimand: they do not measure verifier detection on an identical candidate but rather the end-to-end benefit of verifier-enabled generation and repair. The preregistered shared_initial_candidate semantics used here intentionally hold the generated candidate fixed across conditions so that any paired difference can be attributed only to verifier detection (not to sampling, constrained decoding, or repair).

Deterministic network verification and policy checking. Deterministic analysis tools developed in the networking literature (header-space analysis and related formal checkers) provide precise, semantics-aware invariants for reachability, ACL, and forwarding behavior; these tools are natural verifier candidates for LLM-generated configs because they can conservatively flag violations that imply operational harm [5]. Prior work on real-time policy checking and emulated validation shows how formal analyses can be integrated into testbeds to prevent regressions before deployment. Our canonical verifier (deterministic_netops_validator_v5) was used in the same role: apply deterministic, rule-based checks that compare the generated sequence against manifested workflow constraints (sequence, allowed fields, and protected interfaces) encoded in each task manifest. Where prior literature argues for verifier integration at the architecture level, our contribution is a preregistered paired measurement that quantifies verifier detection power when the generated candidate is held identical across conditions.

Synthesis and gap. Collectively the literature indicates (a) LLMs can produce plausible device commands under constrained settings, (b) multi-agent and tool-augmented systems often propose a verifier role, and (c) GVR and constrained decoding strategies can materially change generation outcomes by construction. What is missing—and what this study targets—is a preregistered, paired, artifact-grounded measurement of verifier detection performance on identical generated candidates: that specific estimand isolates the marginal detection capacity of a verifier decoupled from generation modifications. The degenerate result we observe (zero baseline emulator faults) is thus directly informative: it reveals a design regime—highly constrained synthetic intents and generator behavior—where detection cannot be measured because baseline faults are absent. We use archived traces and validator diagnostics to show how task specification and verifier rule alignment created that regime, and we suggest preregistered alternatives (independent sample arms, repair flows, adversarial task banks) that the literature suggests would expose measurable verifier utility when baseline error prevalence is non-negligible.

III. Methodology

Preregistration and primary estimand. We preregistered and froze study c1-gnr-vv-2026-0001 before observing outcomes. The primary estimand is the paired reduction in undetected operational-fault proportion (paired_success_rate_difference_guarded_minus_baseline). The confirmatory hypothesis (H1) targeted an absolute paired reduction of 0.20 (two-sided alpha = 0.05, power >= 0.80); a sample-size heuristic yielded ≈63 pairs and we preregistered N = 72 pairs (24 per topology-difficulty stratum). The preregistration and analysis plans, including sensitivity analyses and exclusion rules, are part of the archived preregistration record.

Shared_initial_candidate semantics and experimental contract. The execution_contract enforced shared_initial_candidate semantics: for each intent we performed exactly one initial LLM generation and evaluated that identical candidate in two paired conditions. Baseline condition: execute the shared candidate in an isolated emulator and record whether emulator-observed operational faults occur. Guarded condition: run the canonical deterministic verifier on the same candidate; if the verifier flags violations the guarded outcome is recorded as prevented (no executed fault); if the verifier does not flag, execute the identical candidate in the emulator and record the result. By construction, any observed paired difference can only arise from the verifier preventing execution of a candidate that would otherwise have failed at emulation; this isolates detector utility from generator-side resampling or repair effects.

Model, adapter, and verifier configuration (artifact-grounded). The declared model is gpt-5-mini (execution/model_configuration.json), requested via the hosted_netops_gvr_v1 adapter. Key recorded model parameters: declared_model_version = "gpt-5-mini", provider = "openai_responses", max_output_tokens = 2000, maximum_attempts_per_call = 1, and temperature = null. The recorded temperature = null indicates that provider-default runtime sampling semantics were used; we treat this as an archived sampling-parameter uncertainty rather

than an explicit numeric temperature. The canonical verifier is deterministic_netops_validator_v5 (validator_version = "5.0") configured to run the full_intent_constraint_validation rule set. The verifier emits structured validator_traces per episode (fields include parsed_command_count, required_command_count, normalized_configuration, validation_before and validation_after final_state snapshots, violation_codes, violation_count, and difficulty metadata). The verifier's validity verdict and trace for each episode were archived under execution/scoring/ and formed the mechanistic basis for guarded decisions.

Task-bank construction, contamination controls, and stratification. Confirmatory tasks were generated deterministically by deterministic_netops_task_generator_v5. Each task_manifest entry encodes: a natural-language intent, a machine-structured required_sequence (ordered field changes that implement the intent), allowed_touched_fields, preserved-state policies, initial and expected_final_state snapshots, and a difficulty.level tag (medium/hard/very_hard). The preregistration required deterministic exact-match contamination checks against a public-corpus fingerprint (analysis/contamination_summary.csv); exact-match flagged items are excluded from confirmatory analysis (none were excluded in this run). The confirmatory sample followed the preregistered stratification (24 pairs per difficulty stratum) to allow descriptive subgroup inspection.

Execution protocol, retries, and deterministic accounting. The missingness_plan permitted up to two retries (three attempts total) for transient hosted-API errors on generation calls; all retries and provider events were logged. The run started at 2026-08-31T06:56:23.065707Z and completed at 2026-08-31T07:06:28.665518Z (execution manifest). The execution manifest records planned_episode_count = 144, completed_episode_count = 144, failed_episode_count = 0, and model_calls_used = 72 (one shared generation per pair). Provider-call audit (deterministic_reconciliation.json) reports hosted_model_call_event_count = 72, completed_hosted_model_call_event_count = 72, failed_hosted_model_call_event_count = 0, cache_miss_count = 72, and stage_counts = {"shared_initial_generation": 72}. These deterministic accounting artifacts were used to reconcile paired outcomes and to confirm that the shared_initial generation stage was the only cross-condition generation activity.

Scoring, validation rules, and per-episode traces. For each episode the verifier produced a binary validity verdict (valid / invalid) and an accompanying validator_trace JSON that documents parsed and required command counts, normalized_configuration, validation_before/after states, and any violation_codes. The scoring rule deployed was full_intent_constraint_validation (validator enforces required_sequence order, allowed_touched_fields, and preserve_unspecified_state constraints encoded in the task manifest). Per-episode scoring artifacts (execution/scoring/task-*.json) therefore permit audit of why a candidate was accepted or flagged. In the recorded run no verifier flagging occurred for confirmatory pairs (violation_count = 0 across representative episodes), and no automated repair was applied (repair_applied = false in validator_trace fields).

Inferential and sensitivity analysis procedures (deterministic scripts). The primary analysis used the registered paired_binary_analysis_v1 executor to construct the 2×2 contingency table (guarded $\times$ baseline) using binary indicators where an undetected executed fault is 1 and success/no-fault is 0. The prespecified exact inferential test is an exact McNemar two-sided test when discordant pairs exist. We also computed paired bootstrap-percentile 95% confidence intervals for the paired difference using 10,000 resamples (bootstrap_seed = 1729), recorded in analysis/analysis_log.jsonl. Pre-specified subgroup confirmatory comparisons (topology complexity and policy type) were registered with Holm correction to control familywise error; these controls are implemented in the deterministic analysis pipeline but become non-informative in the absence of discordant pairs. Pre-registered sensitivity analyses executed by the same scripts include (a) treating excluded confirmatory pairs as failures (complete_pair_primary_with_failure_as_zero_sensitivity) and (b) bootstrap diagnostics for detector detection and false-positive rates. Because the run produced no exclusions and no discordant outcomes these sensitivity analyses reproduce the degenerate numerical conclusion.

Design-level notes and construct tradeoffs (artifact-linked). The shared_initial_candidate semantics intentionally isolates verifier detection from generator-side sampling; this preserves a clean causal interpretation of verifier-prevention utility but prevents measurements that rely on verifier-triggered resampling or repair. The archived execution/model_configuration.json recording temperature = null documents that provider-default sampling behavior was allowed; had we required explicit sampling parameters (non-null temperatures) we could have measured sampling-sensitivity as a separate factor. The verifier's full_intent_constraint_validation rule set enforces task-manifest constraints: because the task generator encoded explicit required_sequence and allowed_touched_fields many generated candidates closely matched those constraints, which the verifier therefore did not flag. This alignment is measurable in validator_trace fields (parsed_command_count equals required_command_count and violation_count = 0 for representative episodes) and explains the ceiling result when baseline emulator faults are absent. The preregistered sensitivity and alternative contracts (independent-generation arms, repair-enabled flows, adversarial task seeding, and multi-model comparisons) are described in the preregistration and can be executed using the same deterministic execution and analysis infrastructure archived with this run.

IV. Results

Execution and manifest summary. The autonomous pipeline completed without technical failures and with full deterministic accounting. Key run-level figures from

the archived manifests are: planned_episode_count = 144; completed_episode_count = 144; failed_episode_count = 0; model_calls_used = 72; artifact_count = 510. The run-level timestamps (started_at_utc = 2026-08-31T06:56:23.065707+00:00; completed_at_utc = 2026-08-31T07:06:28.665518+00:00) indicate a wall-clock duration of ≈10.1 minutes. analysis/condition_summary.csv (archived) records baseline successes = 72, baseline failures = 0 (success_rate = 1.0) and guarded successes = 72, guarded failures = 0 (success_rate = 1.0).

Paired contingency, inferential outputs, and bootstrap diagnostics. Aggregating across the 72 confirmatory pairs produced the exact paired contingency counts n_11 = 72, n_10 = 0, n_01 = 0, n_00 = 0 (guarded × baseline). The primary estimand paired_success_rate_difference_guarded_minus_baseline = 0.0. The preregistered exact McNemar two-sided test produces p = 1.0 given the absence of discordant pairs. The paired bootstrap-percentile 95% confidence interval for the paired difference (bootstrap_seed = 1729; bootstrap_resamples = 10000; analysis/analysis_log.jsonl) is [0.0, 0.0], reflecting the degenerate sampling distribution when all paired outcomes are concordant. analysis/paired_contingency_table.csv and analysis/condition_summary.csv contain these tabulations and are archived.

Per-episode validator diagnostics and representative exemplars. We inspected archived per-episode scoring JSONs under execution/scoring/ and representative task-manifest entries under manuscript evidence. Across the sampled exemplars the verifier produced no violation flags. Representative excerpts from archived scoring artifacts: - pair-000001 (task-000001): difficulty.level = "medium"; parsed_command_count = 4; required_command_count = 4; validator_version = "5.0"; violation_count = 0; normalized_configuration equals the shared candidate (execution/scoring/task-000001-*.json). - pair-000002 (task-000002): difficulty.level = "hard"; parsed_command_count = 2; required_command_count = 2; violation_count = 0. - pair-000003 (task-000003): difficulty.level = "hard"; parsed_command_count = 6; required_command_count = 6; violation_count = 0. For additional archived representative tasks the task manifests encode required_command_count and difficulty tags (examples: task-000004 required_command_count = 4, difficulty.level = "very_hard"; task-000005 required_command_count = 3, difficulty.level = "very_hard"; task-000006 required_command_count = 6, difficulty.level = "very_hard"). In all archived scoring JSONs for confirmatory pairs the verifier_traces report violation_count = 0 and validation_after.valid = true, demonstrating that the verifier processed each shared candidate and did not flag the candidate as violating the manifest-encoded constraints.

Contamination, missingness, and sensitivity analyses. analysis/contamination_summary.csv records zero exact-match contamination flags for confirmatory pairs. analysis/missingness_summary.csv records complete_pairs = 72 and zeros for failed or unscorable episodes. The preregistered sensitivity analyses (including the conservative treatment that would count excluded pairs as failures and bootstrap diagnostics) were executed by the registered analysis executor; because there were no exclusions and no discordant pairs these sensitivity analyses reproduce the degenerate numeric conclusion (paired difference = 0.0). All analysis artifacts (analysis/paired_contingency_table.csv, analysis/condition_summary.csv, analysis/analysis_log.jsonl) are archived and reference the bootstrap seed and resample count used.

Deterministic reconciliation and provider-call audit. The deterministic_reconciliation.json confirms accounting consistency: completed_episode_count = 144, complete_pair_count = 72, baseline_success_count = 72, guarded_success_count = 72, n_11 = 72, n_10 = 0, n_01 = 0, n_00 = 0, and all_deterministic_consistency_checks_passed = true. Provider-call audit fields show hosted_model_call_event_count = 72, completed_hosted_model_call_event_count = 72, failed_hosted_model_call_event_count = 0, cache_miss_count = 72, and stage_counts = {shared_initial_generation: 72}, consistent with the preregistered shared_initial_candidate semantics and the execution manifest.

Interpretation of the degenerate outcome (evidence-grounded). The complete absence of baseline emulator faults in the archived confirmatory corpus is the proximate cause of the degenerate paired result. The archived artifacts together document measurable contributors: (1) the deterministic task generator produced constrained intents with manifest-encoded required_sequence and allowed_touched_fields that align the generation objective tightly with verifier checks; (2) gpt-5-mini (declared in execution/model_configuration.json) produced configurations that matched those manifest constraints across the sampled tasks; and (3) the deterministic_netops_validator_v5 rule set (full_intent_constraint_validation) enforces the same manifest-encoded sequence and field constraints, producing concordant validity judgments. These archived, machine-verifiable facts explain why the verifier could not reduce executed faults under the shared_initial_candidate contract: there were no executed faults to detect.

V. Discussion

Confirmatory interpretation and boundary conditions. The preregistered paired design with shared_initial_candidate semantics validly measures verifier detection on identical candidates; because the identical generated candidates produced zero emulator faults the verified paired outcome is a ceiling/null result: the verifier could not reduce executed faults that did not occur. The numeric outputs (n_11 = 72; paired difference = 0.0; McNemar p = 1.0; bootstrap CI [0.0,0.0]) follow directly from the preregistered contract and observed data. Reporting this degenerate outcome is important: it documents a reproducible case where the preregistered estimand and data-generating conditions leave no room for the verifier to demonstrate utility.

Artifact-grounded diagnostic factors. Archived artifacts allow concrete attribution of this ceiling outcome to measurable pipeline factors. First, the deterministic task generator (deterministic_netops_task_generator_v5) produced structured manifests with explicit required_sequence and allowed_touched_fields that constrain acceptable configs; these manifest constraints appear in execution/task_manifest.jsonl and increase the probability of a single correct generation. Second, the declared model (gpt-5-mini recorded in execution/model_configuration.json) produced candidates that consistently matched those structured constraints across the synthetic corpus (execution/responses/* files and per-episode shared_initial_candidate_sha256 entries). Third, the verifier's rule set (full_intent_constraint_validation in deterministic_netops_validator_v5) enforces the same required_sequence and field constraints encoded in the manifests; per-episode validator_traces under execution/scoring/ show parsed_command_count = required_command_count and violation_count = 0 for representative tasks. Together these aligned design choices—constrained intents, congruent verifier rules, and a model that produced matching sequences—explain the absence of baseline emulator faults.

Statistical and design implications. From a statistical perspective, a paired confirmatory test that targets a fixed absolute reduction (here preregistered 0.20) requires a nonzero baseline prevalence of executed faults to have power to detect a reduction; when baseline prevalence is zero the paired difference must be zero regardless of verifier performance. This is not a paradoxical failure of the verifier or analysis but an expected algebraic consequence of the estimand under shared_initial_candidate semantics. Practically, therefore, confirmatory designs intended to measure verifier detection should ensure the task bank yields a measurable baseline error rate (e.g., by including ambiguous intents, adversarial cases, or known failure-mode seeds) or should permit condition-specific sampling or repair so that the verifier can alter the executed candidate distribution.

Concrete preregisterable alternatives to detect verifier utility. The archived pipeline supports several clear modifications—each compatible with preregistration—that would permit measuring different aspects of verifier utility without inventing new artifacts here: (1) independent-generation arms: allow separate initial generator calls per condition (baseline vs guarded) so that the verifier (or guarded workflow) can influence candidate distributions via repairs or sampling differences; (2) repair-enabled flows: permit a single deterministic repair call when the verifier flags a violation and then execute the repaired candidate to measure end-to-end improvement in execution outcomes; (3) adversarial/fault-injection task banks: seed tasks with intentionally ambiguous wording, ordering subtleties, or vendor-edge cases to raise baseline error prevalence and probe verifier detection under stress; (4) sampling-parameter sweeps and multi-model comparisons: record explicit non-null temperature and test multiple model versions to quantify model- and sampling-sensitivity; and (5) dedicated false-positive cost arms: measure operational blocking cost by comparing outcomes when verifier blocks flagged candidates versus when blocked candidates are repaired and re-executed. All of these are operationally meaningful, preregistrable changes and can be implemented using the same evidence-recording conventions shown in our run (execution/model_configuration.json, execution/scoring/, analysis/).

Threats to validity revisited with evidence. Internal validity: shared_initial_candidate semantics intentionally isolates verifier detection but prevents verifier-mediated resampling or repair effects; the deterministic provider-call audit (deterministic_reconciliation.json) confirms one shared generation per pair, making the isolation explicit. Construct validity: emulator-executed faults are a conservative surrogate for operational harm but do not capture traffic-dependent timing or vendor-hardware idiosyncrasies—limitations recorded in the manuscript. External validity: the run used a single declared model (gpt-5-mini) and a synthetic task bank; extrapolation to other models, richer real-world task distributions, or production hardware remains limited. Conclusion validity: with zero baseline faults the preregistered power analysis targeted to an absolute 0.20 reduction becomes moot; future confirmatory designs must align expected baseline prevalence with power targets.

Operational implications and measured tradeoffs. The observed ceiling does not imply that verifiers are unnecessary in practice. Instead, it shows that under constrained, high-clarity intents and a verifier rule set that encodes the same constraints, an LLM can produce device-level sequences that pass deterministic checks and emulator execution. In operational settings where intents are incomplete, telemetry is noisy, or vendor dialects vary, verifiers remain a principled guard. Measuring the verifier's net operational value requires explicit, preregistered trade-off quantification: estimate true-positive detection rates on a task bank with non-negligible baseline faults, and quantify verifier false-positive blocking costs (the proportion of flagged-but-safe candidates and operational delay or repair effort). The archived artifacts and deterministic accounting in this study provide a reproducible template to design such targeted, powered follow-on experiments.

VI. Limitations

- Confirmatory ceiling/null outcome: the baseline generator produced zero emulator faults on the preregistered task set, preventing direct measurement of verifier detection benefit.
- Shared_initial_candidate semantics: the design measures verifier effect on the same generated candidate only and does not assess independent per-condition generation, multi-sample generation, or repairs unless explicitly permitted by the contract.
- Single-model scope: experiments used one declared model (gpt-5-mini); results do not imply generality across models or versions.

- Synthetic/emulated tasks: task corpus and emulator executions differ from production devices and live traffic; external validity to production deployments is limited.
- Sampling-parameter uncertainty: execution/model_configuration.json shows temperature = null (sampling parameters unset); null indicates provider-default runtime sampling semantics and is a residual uncertainty recorded in the archived configuration.
- Residual contamination risk: deterministic provenance and exact-match checks were applied, but private training-data contamination of the hosted model cannot be fully ruled out and remains residual uncertainty.

VII. Conclusion

We executed a preregistered paired evaluation (c1-gnr-vv-2026-0001) under shared_initial_candidate semantics to test whether a canonical deterministic verifier reduces the proportion of LLM-generated configurations that would execute with operational faults. For the chosen model (gpt-5-mini), verifier (deterministic_netops_validator_v5), and synthetic/emulated task bank, baseline executions produced zero emulator faults and the verifier did not change outcomes (paired difference = 0.0; bootstrap 95% CI [0.0,0.0]; exact McNemar p = 1.0). This artifact-grounded null/ceiling outcome is internally consistent with the preregistered design: when identical generated candidates produce no executed faults a verifier cannot reduce executed faults under the shared_initial_candidate contract. Measuring verifier utility under realistic error regimes requires preregistered contracts that permit independent or multiple generator samples, repair-enabled flows, adversarial task sets, and multi-model comparisons. The archived artifacts (responses, task manifests, validator traces, execution manifest, and analysis logs) provide a reproducible baseline and a template for those follow-on confirmatory designs and power calculations.

Disclosure Statement

Autonomy and artifacts: This study was executed autonomously after an autonomy lock. Human role: a Research Director selected the topic family and approved the immutable initial master prompt prior to the autonomy lock; no human manually edited technical manuscript content after the lock. Models and tools: initial generation used gpt-5-mini (declared_model_version recorded in execution/model_configuration.json) orchestrated via the hosted_netops_gvr_v1 adapter; deterministic_netops_validator_v5 (validator_version = "5.0") served as the canonical verifier; an isolated emulator executed candidate configurations. Preregistration: study c1-gnr-vv-2026-0001 was preregistered and frozen before observing outcomes. Immutable master prompt reference: provenance/master_prompt.txt (SHA-256: 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba). Key archived artifacts include execution/raw_results.jsonl, execution/task_manifest.jsonl, execution/model_configuration.json, analysis/paired_contingency_table.csv, analysis/condition_summary.csv, deterministic_reconciliation.json, and per-episode validator traces under execution/scoring/. The model configuration records temperature = null (provider-default sampling semantics archived in execution/model_configuration.json). Provider-call audit: hosted_model_call_event_count = 72. No human manual manuscript editing or content insertion occurred after the autonomy lock.

References

[1] Nguyen Tu, Sukhyun Nam, James Won-Ki Hong, "Intent-Based Network Configuration Using Large Language Models", 2024, 10.1002/nem.2313

[2] Zhaodong Wang, Samuel Lin, Guanqing Yan, Soudeh Ghorbani, Minlan Yu, Jiawei Zhou, Nathan Hu, Lopa Baruah, Sam Peters, Srikanth Kamath, Jian Yang, Ying Zhang, "Intent-Driven Network Management with Multi-Agent LLMs: The Confucius Framework", 2025, 10.1145/3718958.3750537

[3] Rafael Cadenas, "Convergent Correctness in Stochastic Code Generation: A Generate-Verify-Repair Architecture for Deterministic Validation of Autonomous AI Coding Agents in Regulated Environments", 2026, 10.2139/ssrn.6754899

[4] http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.300.8659